

INSTITUTO FEDERAL DO PIAUÍ - CAMPUS PAULISTANA
DISCIPLINA: INTELIGÊNCIA ARTIFICIAL
PROFESSORA: MAÍLA DE LIMA CLARO

Relatório Técnico

Classificação de Imagens no CIFAR-100 com Rede Neural Convolutacional (CNN)
(WRN-34-10-XL)

Aluno: Felipe Pereira Coelho

Trabalho Prático: Redes Neurais Convolucionais (CNNs)

1. Introdução

Este trabalho tem como objetivo desenvolver uma Rede Neural Convolutacional (CNN) capaz de classificar imagens da base de dados CIFAR-100 com a maior precisão possível, aplicando conceitos de Deep Learning, otimização de hiperparâmetros e técnicas de redução de overfitting.

A base de dados CIFAR-100 contém 60.000 imagens coloridas de tamanho 32x32 pixels, distribuídas em 100 classes diferentes, com 500 imagens de treino e 100 imagens de teste por classe. A grande quantidade de classes e a baixa resolução das imagens tornam esse problema significativamente mais desafiador do que conjuntos de dados com menos categorias, como o CIFAR-10, exigindo uma arquitetura robusta e técnicas de regularização bem ajustadas para evitar overfitting.

Para este trabalho foi desenvolvida, a partir do zero e sem o uso de modelos ou pesos pré-treinados, uma arquitetura baseada no conceito de Wide Residual Network (WRN), combinada com diversas técnicas modernas de data augmentation, regularização e ajuste de hiperparâmetros. Após um extenso processo iterativo de treinamentos e ajustes, o modelo final alcançou 83,58% de acurácia top-1 no conjunto de teste.

2. Arquitetura da CNN

A arquitetura utilizada é uma Wide Residual Network, construída inteiramente do zero seguindo o conceito apresentado por Zagoruyko e Komodakis (2016), porém com adaptações próprias na profundidade, largura e número de estágios da rede. Por incorporar um quarto estágio de blocos residuais (ausente na WRN original, que possui apenas três grupos), esta variação será referida ao longo deste relatório como WRN-34-10-XL.

2.1 Estrutura Geral

A rede segue o seguinte fluxo:

- Camada convolutacional inicial (Conv2D 3x3, 16 filtros) para extração de features de baixo nível;
- Quatro grupos de blocos residuais, com 5 blocos cada (depth=34), e número de filtros crescente: 160 → 320 → 640 → 1280;
- Batch Normalization e ativação ReLU ao final dos blocos residuais;
- Global Average Pooling, que reduz cada mapa de características a um único valor por canal, eliminando a necessidade de uma camada Flatten com grande número de parâmetros;
- Camada densa (Dense) final com 100 neurônios (um por classe) e ativação Softmax para gerar as probabilidades de classificação.

2.2 Blocos Residuais

Cada bloco residual segue a estrutura pré-ativação (BatchNorm → ReLU → Conv), que facilita o fluxo do gradiente em redes profundas:

- BatchNormalization → ReLU → Conv2D 3x3
- BatchNormalization → ReLU → Dropout → Conv2D 3x3
- Soma (skip connection) com a entrada do bloco, usando uma convolução 1x1 no atalho (shortcut) sempre que há mudança de dimensão ou número de filtros.

O downsampling espacial entre os grupos de blocos é feito por meio de convoluções com stride=2 (e não por camadas de MaxPooling2D), abordagem padrão em ResNets modernas, pois permite que os pesos aprendam a reduzir a resolução em vez de simplesmente descartar informação.

2.3 Quantidade de Camadas e Filtros

Característica	Valor
Nome da arquitetura	WRN-34-10-XL
Profundidade (depth)	34
Largura (width)	10
Blocos por grupo	5
Grupos de blocos	4 (160, 320, 640, 1280 filtros)
Total de parâmetros	187.258.196 ($\approx$ 187,3M)
Função de ativação	ReLU (camadas ocultas) / Softmax (saída)

A escolha por quatro grupos de blocos (em vez dos três grupos da WRN original) e por um quarto estágio com 1280 filtros foi uma decisão arquitetural deliberada: aumenta a capacidade representacional da rede para lidar com as 100 classes do CIFAR-100, ao custo de mais parâmetros e tempo de treino. Essa escolha se mostrou eficaz na prática, sustentando os ganhos de acurácia observados ao longo dos experimentos.

3. Hiperparâmetros

Os hiperparâmetros finais foram definidos por meio de um processo iterativo de múltiplos treinamentos, ajustando um parâmetro de cada vez e observando o impacto na acurácia de validação.

Hiperparâmetro	Valor
Otimizador	SGD com Momentum (0.9) e Nesterov
Taxa de aprendizado inicial	0,2
Warmup	5 épocas (crescimento linear de 0,04 a 0,2)
Schedule (step decay)	Decaimento $\times 0,2$ nas épocas 65, 95, 130 e 155
Épocas	180
Batch size	256
Dropout	0,2
Label Smoothing	0,1
Weight Decay (L2)	1×10^{-4}
Precisão de treino	Mixed Precision (float16)

O schedule de learning rate por step decay (em vez de cosine annealing) foi escolhido após testes comparativos: o decaimento em pontos específicos, calibrados a partir da observação das curvas de validação de treinos anteriores, apresentou resultados superiores e mais estáveis do que o decaimento contínuo. O atraso do primeiro decaimento até a época 65 permitiu que o modelo explorasse mais o espaço de busca com LR alto antes de refinar os pesos.

4. Técnicas Utilizadas

4.1 Data Augmentation

- Random Crop com padding de 4 pixels (reflect padding), preservando informações de borda;
- Flip horizontal aleatório;
- AutoAugment simplificado: aplicação de duas operações aleatórias por imagem entre AutoContrast, Brightness, Sharpness, Equalize, Solarize, Color, Posterize e Hue;
- Random Erasing: apaga aleatoriamente uma região retangular da imagem (com probabilidade 0,5), substituindo-a por ruído;
- MixUp: combina linearmente pares de imagens e seus rótulos;
- CutMix: recorta uma região de uma imagem e a insere em outra, ajustando os rótulos proporcionalmente à área trocada. MixUp e CutMix são alternados aleatoriamente (50%/50%) a cada batch.

4.2 Regularização

- Dropout (0,2) dentro de cada bloco residual;
- Batch Normalization antes de cada ativação ReLU (estrutura pré-ativação);
- Regularização L2 (weight decay) em todas as camadas convolucionais e na camada densa;
- Label Smoothing (0,1) na função de perda, suavizando os rótulos para evitar overconfidence;
- Early Stopping (paciência de 30 épocas, monitorando val_accuracy, com restauração dos melhores pesos).

4.3 Otimização e Infraestrutura

- SGD com momentum e Nesterov, com schedule de learning rate por step decay e warmup;
- Mixed Precision (float16) para acelerar o treinamento em GPU, mantendo a camada de saída em float32 por estabilidade numérica;
- XLA (jit_compile) para otimização do grafo computacional;
- ModelCheckpoint salvando o melhor modelo conforme a acurácia de validação.

5. Resultados e Discussão

O modelo final foi treinado por 180 épocas e avaliado no conjunto de teste do CIFAR-100, obtendo os seguintes resultados:

Métrica	Valor
Loss (teste)	1,6720
Top-1 Accuracy	83,58%
Top-5 Accuracy	96,34%
Macro Precision	83,78%
Macro Recall	83,58%
Macro F1-Score	83,58%

5.1 Gráfico de Treinamento

Curvas de Aprendizado — WRN-28-10 CIFAR-100

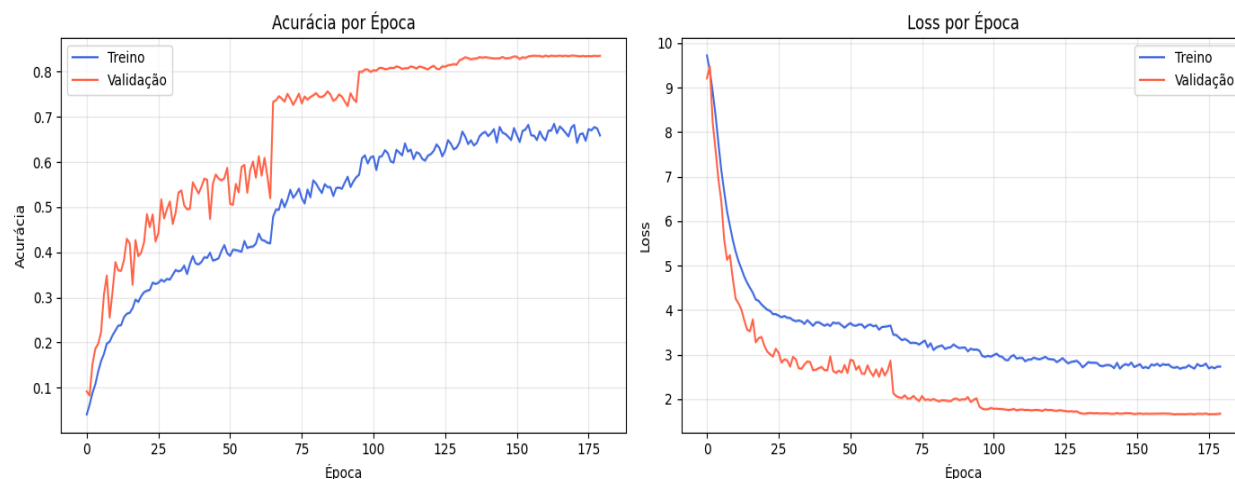


Figura 1 - Curvas de acurácia e loss (treino x validação) ao longo das 180 épocas.

As curvas evidenciam claramente o efeito do step decay: os saltos abruptos de acurácia nas épocas 66, 96 e 131 correspondem exatamente às reduções programadas da taxa de aprendizado, permitindo ao modelo refinar os pesos após cada queda do learning rate. É possível notar que a acurácia de validação se manteve consistentemente acima da acurácia de treino durante quase todo o processo — comportamento atípico à primeira vista, mas esperado neste caso, pois as técnicas de data augmentation aplicadas apenas no treino (MixUp, CutMix, AutoAugment e Random Erasing) tornam a tarefa de treino artificialmente mais difícil que a de validação, que utiliza apenas as imagens originais.

A curva de loss de validação se estabiliza por volta da época 155, sugerindo que o modelo se aproximou de sua capacidade máxima sob esta configuração de hiperparâmetros e que extensões adicionais de treinamento provavelmente trariam ganhos marginais.

5.2 Matriz de Confusão

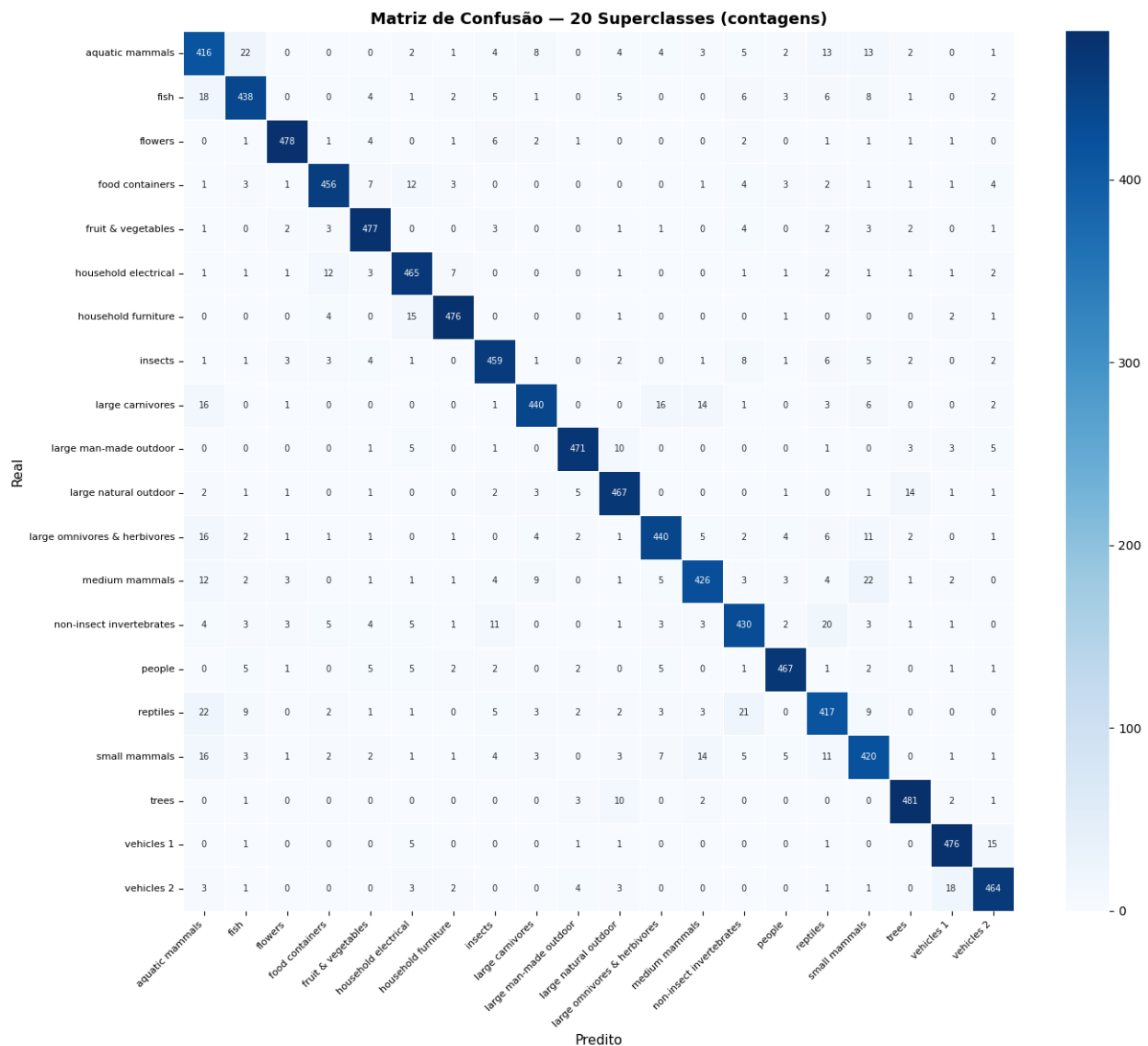


Figura 2 - Matriz de confusão agregada por superclasse (20 superclasses do CIFAR-100).

Como o CIFAR-100 possui 100 classes finas, a matriz de confusão completa seria pouco legível; por isso optou-se por agregá-la nas 20 superclasses oficiais do dataset. A diagonal principal é fortemente dominante em praticamente todas as superclasses, com a maioria delas acima de 90% de acerto, o que indica que o modelo aprendeu representações visuais consistentes e bem separadas entre as categorias de alto nível.

As principais confusões observadas ocorrem entre superclasses visualmente ou semanticamente próximas, como large carnivores e large omnivores & herbivores, e entre medium mammals, small mammals e non-insect invertebrates — categorias que compartilham texturas, formas e contextos de fundo semelhantes em imagens de 32x32 pixels.

5.3 Acurácia por Classe

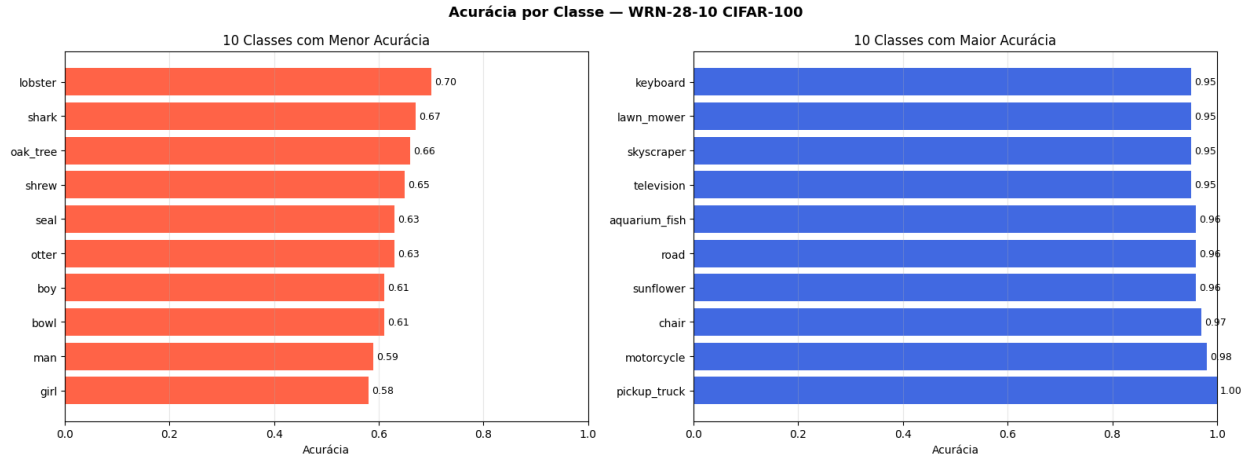


Figura 3 - Dez classes com menor e maior acurácia individual.

As classes com menor desempenho foram girl (58%), man (59%), bowl (61%) e boy (61%), todas pertencentes às superclasses people e household. Esse resultado é coerente com a literatura sobre o CIFAR-100: imagens de pessoas em baixa resolução tendem a ser visualmente ambíguas (poses, roupas e enquadramentos muito variados), e objetos genéricos como bowl podem ser confundidos com outros utensílios de formato semelhante.

Já as classes com melhor desempenho — pickup_truck (100%), motorcycle (98%), chair (97%) e sunflower (96%) — são objetos com formas e texturas bastante distintivas e contextos visuais característicos, o que facilita sua separação pelo modelo mesmo em baixa resolução.

5.4 Exemplos de Classificações

Exemplos de classificações corretas

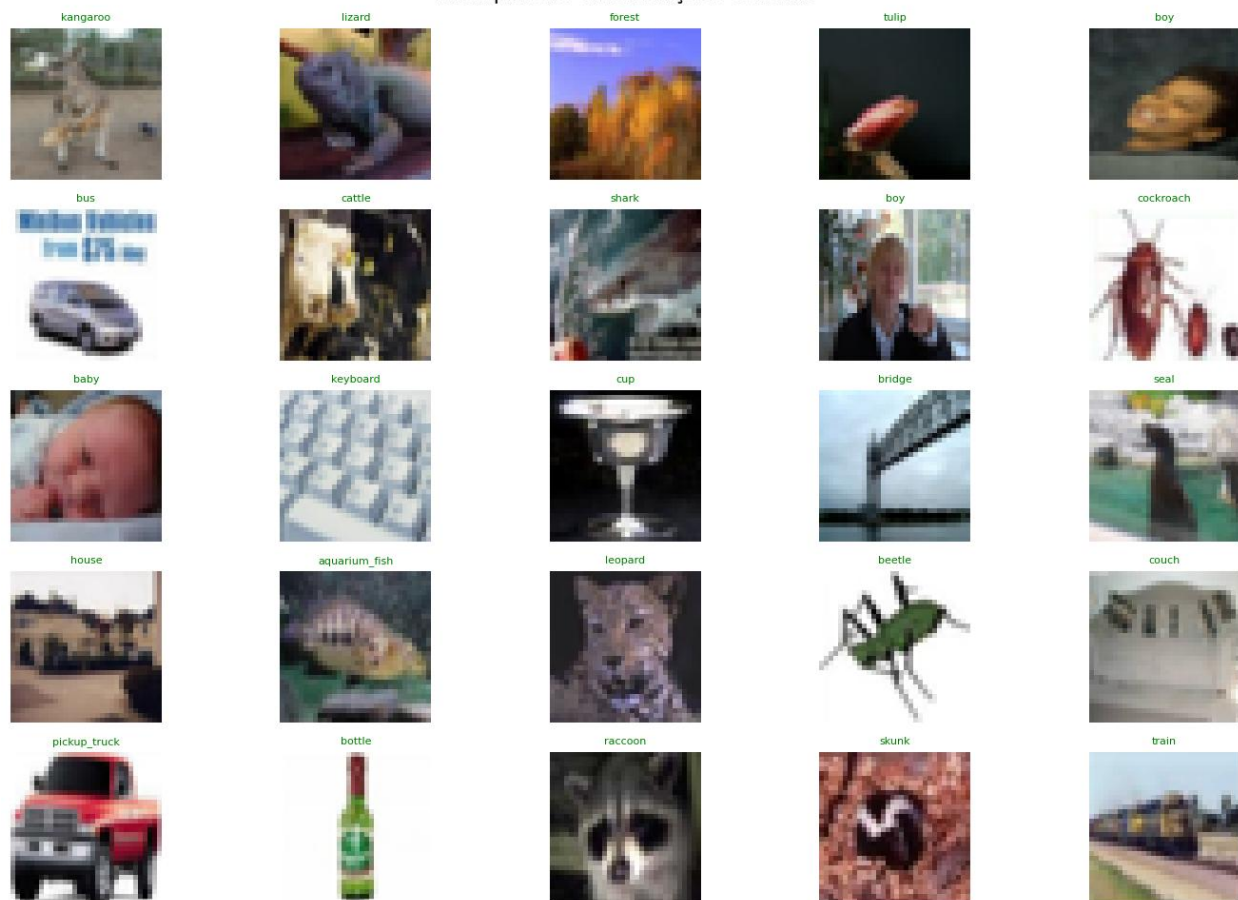


Figura 4 - Exemplos de classificações corretas.

Classificações incorretas

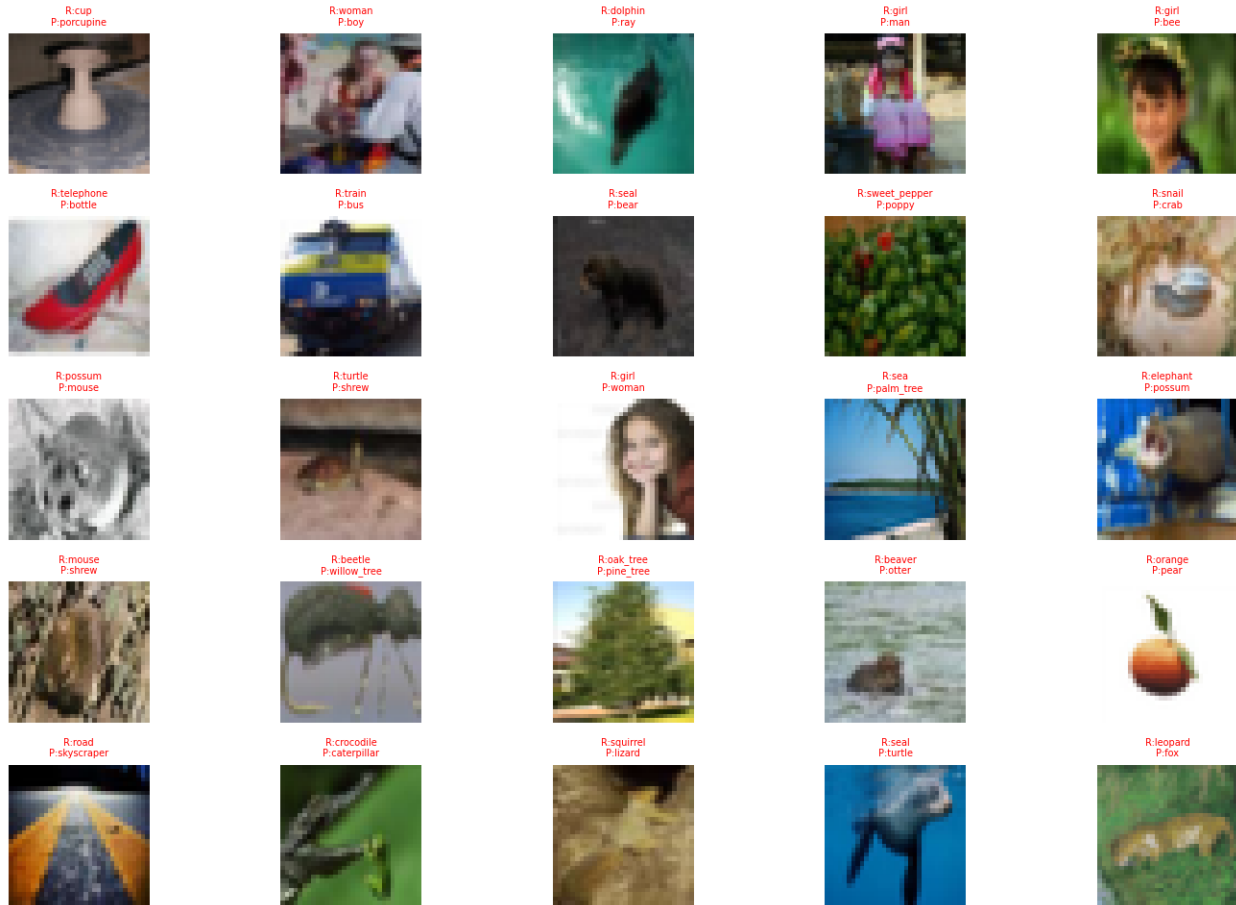


Figura 5 - Exemplos de classificações incorretas.

Analisando os erros, observa-se um padrão recorrente: a maior parte das confusões ocorre entre classes com alta similaridade visual ou semântica, como dolphin/ray, seal/bear, oak_tree/pine_tree e crocodile/caterpillar. Em vários casos, mesmo um observador humano teria dificuldade em distinguir a classe correta apenas pela imagem em baixa resolução, o que sugere que o modelo está operando próximo do limite teórico de desempenho imposto pela qualidade dos dados, e não por uma deficiência fundamental da arquitetura ou do treinamento.

5.5 Evolução ao Longo dos Experimentos

O resultado final de 83,58% foi alcançado após um processo iterativo extenso, com diversos ajustes de hiperparâmetros, schedule de learning rate, dropout e regularização. A tabela a seguir resume a evolução do projeto:

Configuração principal	Épocas	Dropout / WD	Acurácia
Schedule [40,80,100]	120	0,3 / $1e-4$	77,40%
Schedule [65,95,130]	150	0,3 / $1e-4$	82,51%
Schedule [65,95,130,155]	180	0,2 / $5e-4$	83,35%
Schedule [65,95,130,155] (final)	180	0,2 / $1e-4$	83,58%

Dentre as descobertas mais relevantes desse processo iterativo, destacam-se: (1) o atraso do primeiro decaimento de learning rate até a época 65 (em vez de épocas mais cedo, como 40 ou 50) trouxe ganhos consistentes em todos os testes, pois permitiu que o modelo explorasse mais o espaço de pesos antes do refinamento; (2) a redução do dropout de 0,3 para 0,2 melhorou o resultado, pois a combinação de MixUp, CutMix, AutoAugment e Random Erasing já fornece regularização suficiente via dados, tornando dropout adicional excessivo; (3) contraintuitivamente, um weight decay menor (1×10^{-4}) superou um valor maior (5×10^{-4}), reforçando a mesma conclusão sobre regularização excessiva; (4) uma tentativa de aumentar a resolução de entrada para 64x64 pixels (upsampling) mostrou-se computacionalmente inviável dentro do limite de 12 horas contínuas de GPU oferecido pela plataforma Kaggle, sendo abandonada.

6. Conclusão

O modelo desenvolvido — uma WRN-34-10-XL (Wide Residual Network de 34 camadas, com quatro estágios de blocos residuais), construída integralmente do zero, sem uso de pesos pré-treinados — atingiu 83,58% de acurácia top-1 e 96,34% de acurácia top-5 no conjunto de teste do CIFAR-100, resultado obtido por meio de um processo extenso e sistemático de ajuste de arquitetura, hiperparâmetros e técnicas de regularização e data augmentation.

O trabalho demonstrou, na prática, o papel central de fatores muitas vezes subestimados em comparação à arquitetura em si: o desenho cuidadoso do schedule de learning rate (em particular, o momento exato dos decaimentos) e o balanceamento entre as diferentes formas de regularização (dropout, weight decay e data augmentation) tiveram impacto mensurável e significativo na acurácia final, por vezes superior ao impacto de mudanças estruturais na rede.

Como direções futuras, técnicas como Test-Time Augmentation (TTA), Exponential Moving Average (EMA) dos pesos e Stochastic Weight Averaging (SWA) poderiam ser exploradas para extrair ganhos adicionais sem necessidade de retreinamento completo, assim como o aumento da resolução de entrada (upsampling, aumentando para 64x64), que se mostrou promissor mas computacionalmente inviável dentro das restrições de tempo e hardware disponíveis para este trabalho.